SOMAIYA VIDYAVIHAR UNIVERSITY
K J Somaiya College of Engineering

A Comparative Study of How Hyperparameters Affect PPO and SAC Agents for Autonomous Lunar Landing

Reinforcement Learning Lab CA · Mini Project

By
Girish Chaudhari (16010123125)
Atharva Patil (16010124814)
Gurkaran Ahuja (16010123129)

The problem: land on planets the agent cannot see

A lander must touch down gently with a main engine and two side engines. Each landing is on a different planet whose gravity, wind and turbulence are hidden from the agent, so one policy must work on all of them.

2019

Chandrayaan-2's lander was lost in its final descent

2023

Chandrayaan-3 soft-landed near the lunar south pole

0.17 g

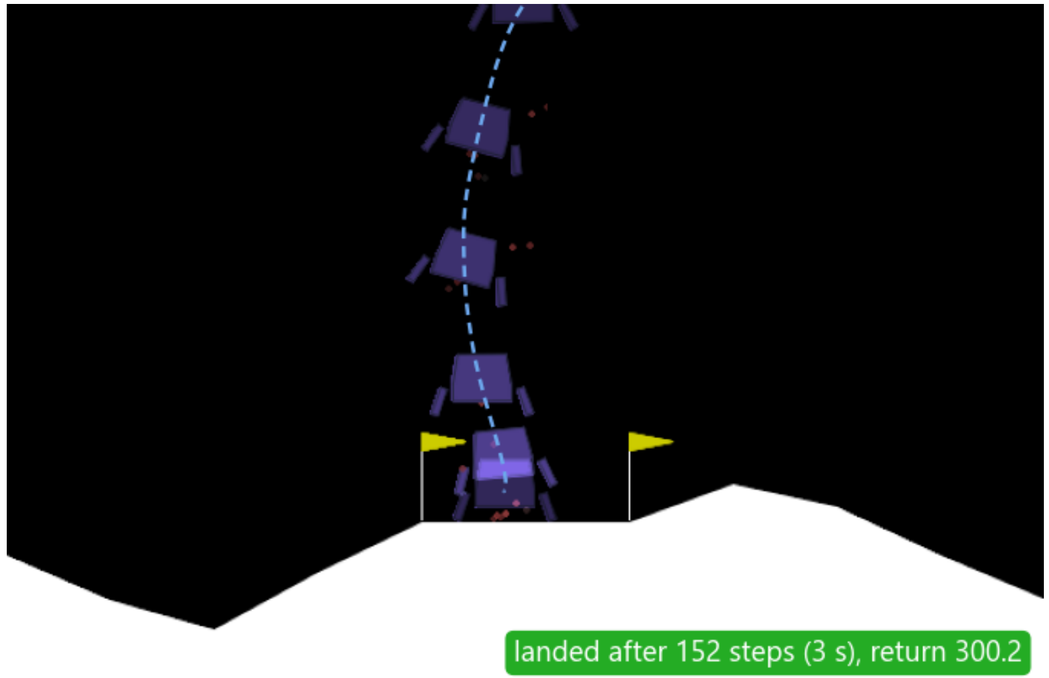
on the Moon, 0.38 g on Mars. Our unseen low-gravity planets: 0.2 to 0.4 g

Q1 Which setting matters most?

the algorithm, learning rate, network size or training length?

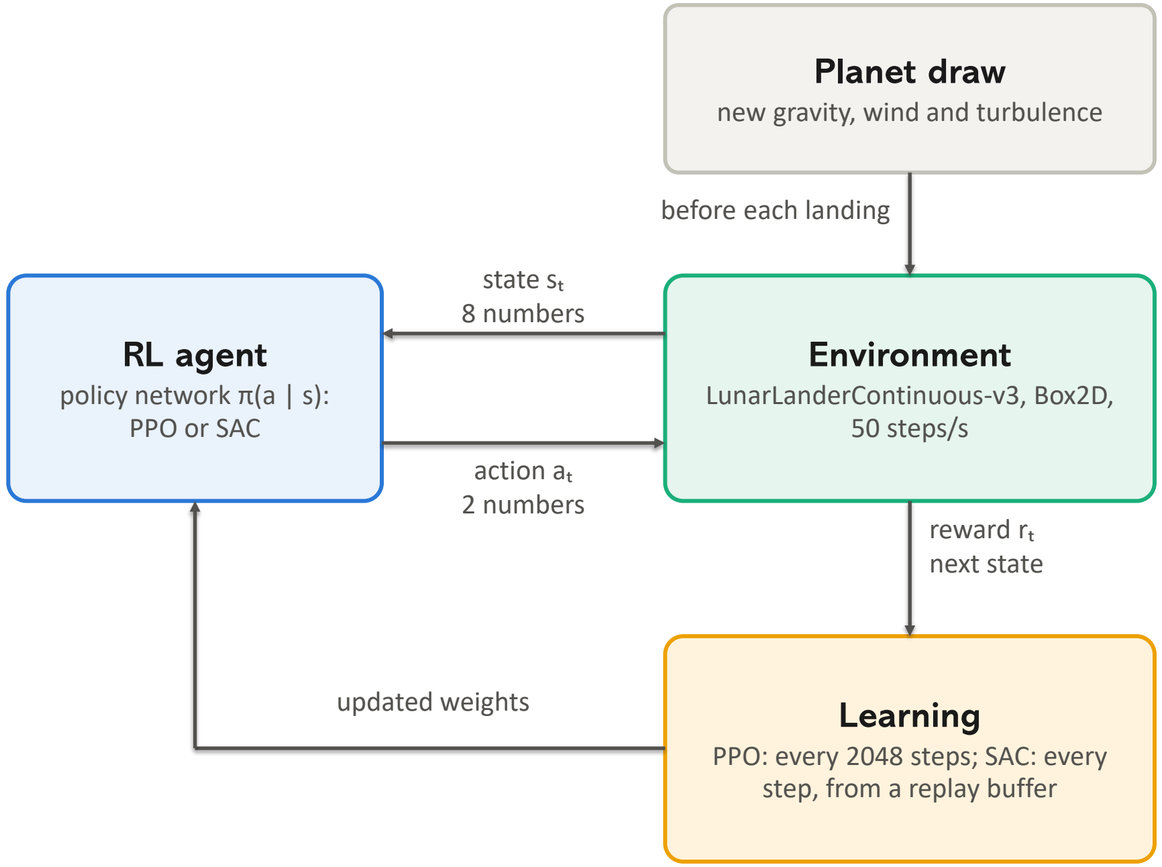
Q2 Where does performance drop off?

when does more training or a higher learning rate make an agent worse?



The best SAC agent landing in heavy wind (wind 18.8, turbulence 1.75): 3 s, return 300.

RL formulation: 8 numbers in, 2 engine commands out



Component	Our task
Environment	LunarLanderContinuous-v3 (Gymnasium, Box2D, 50 steps/s); a new random planet each landing
State	8 numbers: position, velocity, angle, angular velocity, 2 leg contacts. No gravity or wind: partially observed
Action	2 numbers in [−1, 1]: main engine and side engines
Reward	Shaping for getting closer, slower and level, +10 per leg down, a fuel cost; +100 at rest, −100 for a crash. 200 = solved
Discount γ	0.99
Episode ends	At rest, crash, off-screen, or 1000 steps (20 s)

Why reinforcement learning?

- No dataset of correct thrust, but a simulator allows safe practice
- The decisive reward comes at the end (credit assignment)
- One policy can cover a whole range of planets

State, action and reward in detail

#	State variable	Range
1–2	x, y position relative to the pad	−2.5 to 2.5 (normally −1 to 1)
3–4	x, y velocity	−10 to 10
5	angle (radians)	−2π to 2π
6	angular velocity	−10 to 10
7–8	left, right leg on the ground	0 or 1

Gravity and wind are not in the state: the agent has to infer the planet.

Action	Effect
a_0 main engine	off at ≤ 0; above 0, throttle 50–100%
a_1 side engines	off from −0.5 to 0.5; below −0.5 the left engine fires, above 0.5 the right; throttle 50–100%

REWARD PER STEP

$$r_t = \text{shaping}_t - \text{shaping}_{t-1} - 0.3 \cdot \text{main} - 0.03 \cdot \text{side}$$

$$\text{shaping} = -100 \cdot \text{distance} - 100 \cdot \text{speed} - 100 \cdot |\text{angle}| + 10 \cdot (\text{legs down})$$

- Getting closer, slower and more level raises shaping, so it is rewarded every step
- Fuel: −0.3 per step at full main throttle, −0.03 for a side engine

AT THE END OF A LANDING

+100 comes to rest

−100 crash, or off-screen ($|x| \geq 1$)

A landing that totals 200 or more counts as solved.

Training planets vs test planets

Training
a new planet before every landing

gravity **N(−10, 1)**

wind **|N(0, 7)|, at most 20**

turbulence **|N(0, 0.7)|, at most 2**

Ideal

gravity **−10** wind **0** turb. **0**

seen in training

Moderate wind

gravity **−10** wind **5–10** turb. **0.5–1**

seen in training

Heavy wind

gravity **−10** wind **15–20** turb. **1.5–2**

rarely seen (3% of landings)

Heavy gravity

gravity **−11 to −11.9** wind **0** turb. **0**

seen (14% of landings)

Low gravity

gravity **−2 to −4** wind **0** turb. **0**

never seen in training

Random mix

gravity **−2 to −11.9** wind **0–20** turb. **0–2**

partly seen

300 fixed test planets: 50 per condition, drawn once (seed 2026). Every agent makes one deterministic landing on each.

Two algorithms: PPO vs SAC

PPO Proximal Policy Optimization

$$L(\theta) = E[\ \min(\ r_t(\theta)\hat{A}_t\ ,\ clip(r_t(\theta),\ 1-\varepsilon,\ 1+\varepsilon)\ \hat{A}_t\ )\ ]$$

- **On-policy:** learns only from fresh experience
- **Explores** by adding noise to its actions
- **Fast to train:** about 0.7 h per run
- **Sensitive:** high learning rates broke big networks
- **Best agent:** 220 return, 85% landed

SAC Soft Actor-Critic

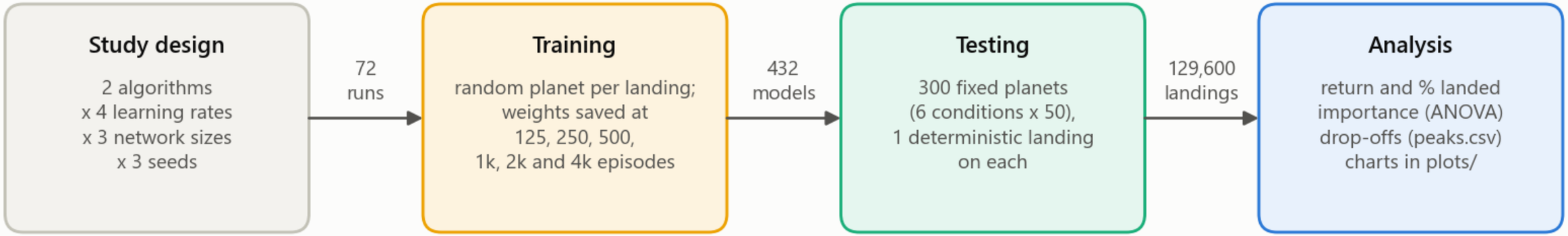
$$J(\pi) = \Sigma_t\ E[\ r_t + \alpha\ H(\pi(\cdot \mid s_t))\ ]$$

- **Off-policy:** keeps and reuses past experience
- **Explores** because acting randomly is rewarded
- **Slow to train:** about 5–9 h per run
- **Robust:** never collapsed in our runs
- **Best agent:** 275 return, 96% landed

Why not DQN? It needs discrete actions; the throttles are continuous. Both use Stable-Baselines3 defaults except learning rate and network size.

Experiment design: 72 runs, 432 agents, 129,600 landings

ALGORITHM	LEARNING RATE	NETWORK	SEEDS	SAVED AFTER (EPISODES)
PPO, SAC	1e-4, 3e-4, 1e-3, 3e-3	16x16, 64x64, 256x256	0, 1, 2	125, 250, 500, 1k, 2k, 4k



28 h
to train all 72 runs on one laptop (271 run-hours)

11
runs at once, one CPU thread each

7–13×
longer per SAC run than per PPO run

≈10 min
to test all 432 agents

How the code works: three commands

1 **72 runs, 11 at a time**

```
python driver.py train
```

trainer.train() wraps the simulator in GaussPlanetWrapper: a new planet each landing. StageSaver saves the weights at the 6 stages, so a stopped run resumes.

2 **432 agents × 300 planets**

```
python driver.py test --log
```

testing.test() flies all 300 landers side by side with one batched, deterministic network call per step: 3.4× faster.

3 **charts and analysis**

```
python driver.py plots
```

plots.make_plots() draws the charts, runs the analysis of variance (variance_shares()) and finds where each setting peaks.

Our code vs the library

PPO and SAC are Stable-Baselines3's. We wrote the harness: planet randomization, staged saves with an exact resume, batched testing and the analysis.

`names.py` holds names and paths; `diagram.py` draws the diagrams.

Engineering challenges

- A Windows Update restart killed the first attempt, so runs save at every stage, write files atomically and resume exactly
- The GPU was no faster than one CPU thread (30–80 µs per kernel launch), so we trained on 11 CPU workers

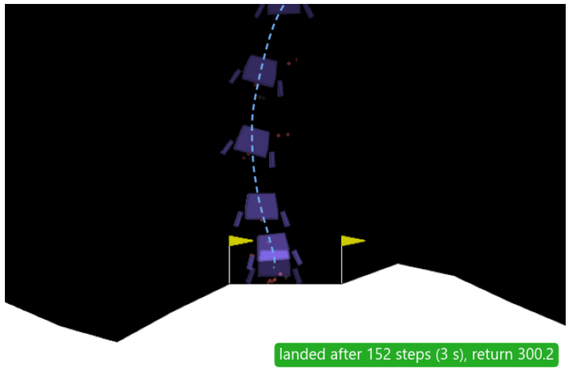
Demo: the best agent on all 300 test planets

SAC 1: SAC, 256x256, lr 3e-3, seed 0, after 4000 episodes

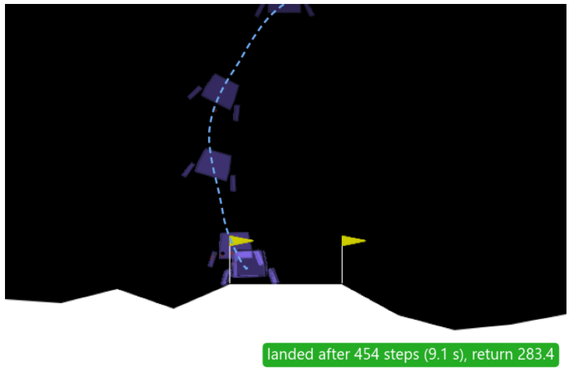
Condition	Mean return	Landed	Crashed	Timeout
ideal	282.6	100%	0%	0%
moderate wind	288.8	100%	0%	0%
heavy wind	272.3	94%	4%	2%
heavy gravity	282.7	100%	0%	0%
low gravity	259.1	90%	6%	4%
random mix	273.8	98%	2%	0%
all 300	**276.6**	**97%**	**2%**	**1%**

Watch it live:

```
python testing.py resources/sac/sac_lr3e-3_net256x256_seed0_ep4000.zip
    --render --conditions heavy_wind
```



Heavy wind (18.8): lands in 3 s



Low gravity (−2.73), never trained on: hovers, lands in 9.1 s

- Reliable inside its training range: 98–100% landed in ideal, both winds and heavy gravity
- Low gravity is its weak spot: 90% landed for this seed, slower and hovering
- Every watched landing matches its logged test result

Best settings: SAC wins, and SAC 4 is the bargain

SAC 1 **275.1**

mean return · 96.2% landed

SAC, 256x256, lr 3e-3, 4000 episodes

PPO 1 **220.4**

mean return · 85.0% landed

PPO, 256x256, lr 1e-4, 4000 episodes

SAC 4 **264.3**

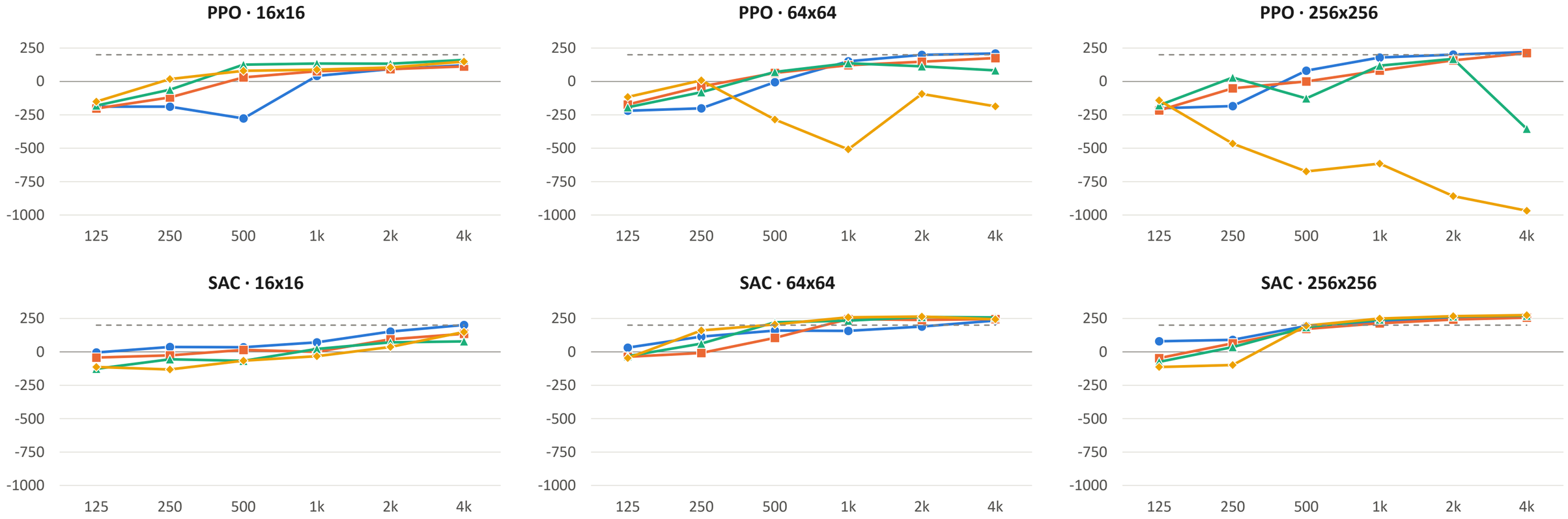
in ≈2.0–2.6 h, vs 9.1–10.1 h for SAC 3 (264.8)

SAC, 64x64, lr 3e-3, 2000 episodes

Rank	Setting	Mean return	Seed range	Landed %	Steps per run	Hours per run
SAC 1	SAC, 256x256, lr 3e-3	275.1	273.8–276.6	96.2	0.83–0.92 M	8.1–9.8
SAC 2	SAC, 256x256, lr 1e-3	271.4	262.7–277.9	96.0	0.90–0.94 M	8.8–9.1
SAC 3	SAC, 256x256, lr 1e-4	264.8	258.0–270.3	95.3	1.10–1.24 M	9.1–10.1
SAC 4	SAC, 64x64, lr 3e-3 (2000 episodes)	264.3	255.1–272.8	93.0	0.62–0.77 M	≈2.0–2.6
PPO 1	PPO, 256x256, lr 1e-4	220.4	214.9–223.7	85.0	1.71–1.88 M	0.8–1.0
PPO 2	PPO, 256x256, lr 3e-4	212.3	197.0–234.2	85.5	1.53–1.56 M	0.8–0.9
PPO 3	PPO, 64x64, lr 1e-4	209.3	191.6–229.5	79.5	1.76–1.91 M	0.8–0.9

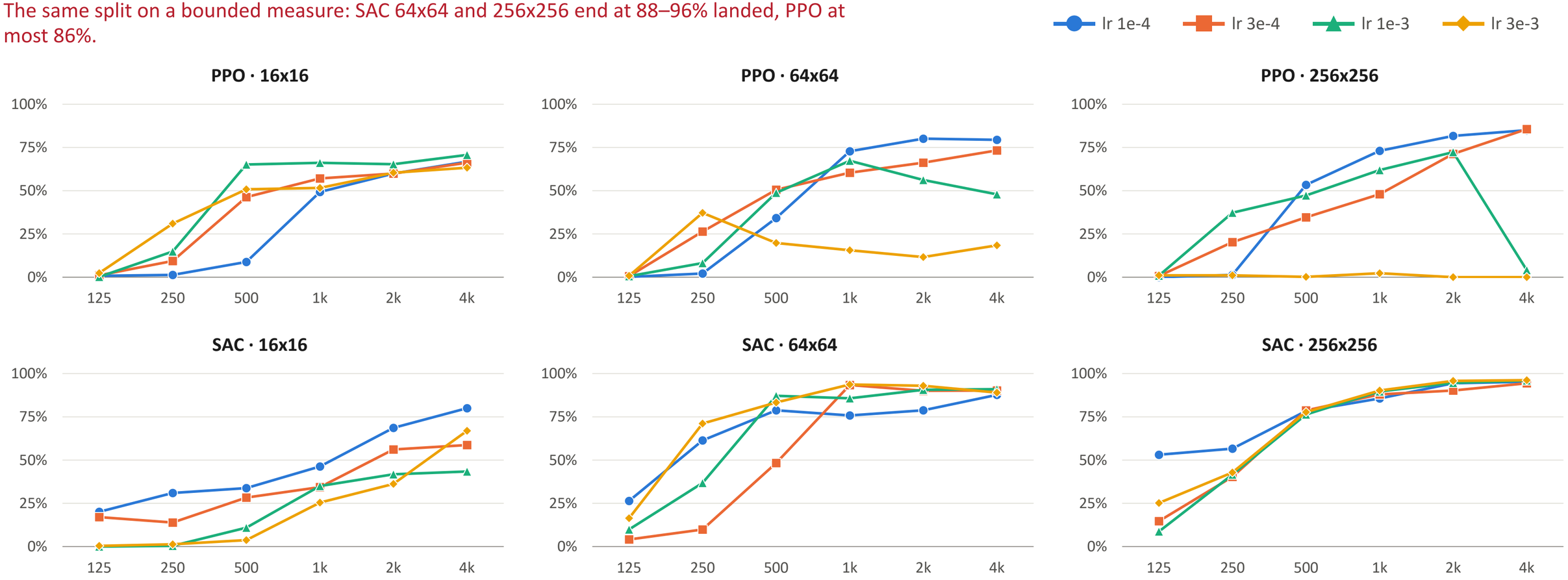
Test return vs training episodes, per learning rate

PPO with lr ≥ 1e-3 on 64x64 or 256x256 collapses; SAC above 16x16 improves at every learning rate. Each line: mean of 3 seeds.



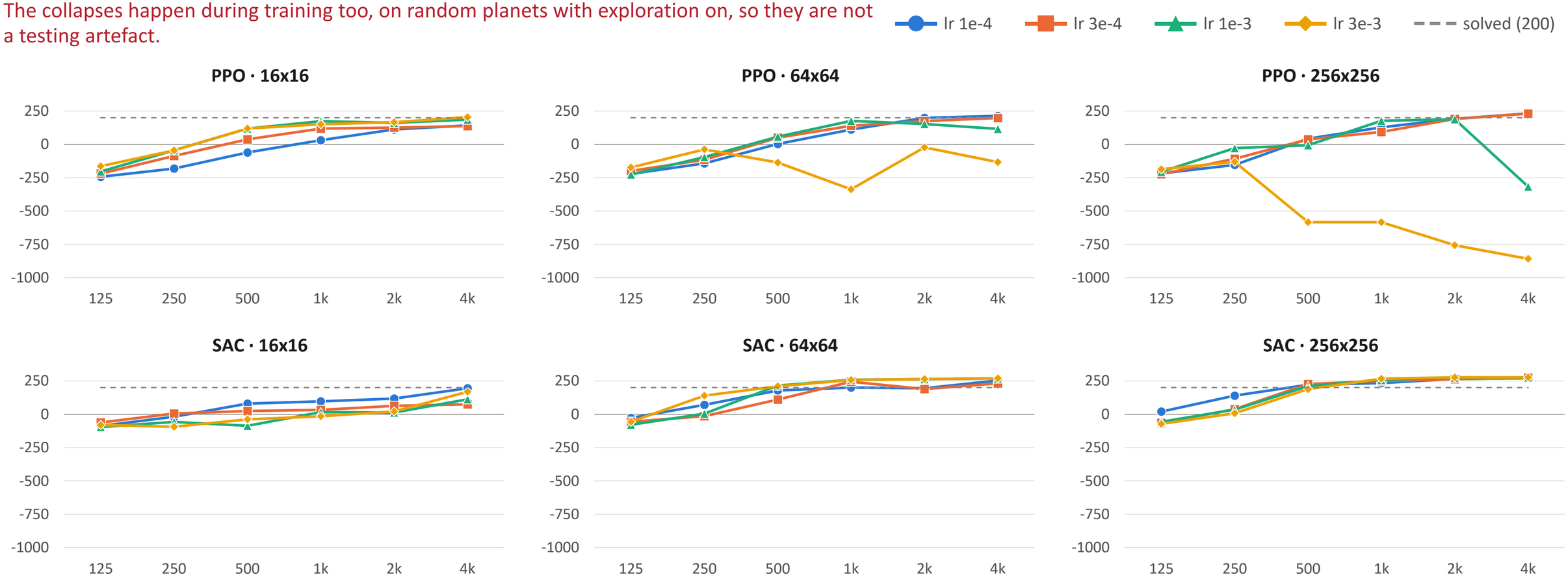
Landing success vs training episodes, per learning rate

The same split on a bounded measure: SAC 64x64 and 256x256 end at 88–96% landed, PPO at most 86%.



Training return vs training episodes, per learning rate

The collapses happen during training too, on random planets with exploration on, so they are not a testing artefact.



F1 · SAC wins per episode and per sample, not per hour

MEAN TEST RETURN

SAC 1 275.1

PPO 1 220.4

LANDED

SAC 1 96.2%

PPO 1 85.0%

ENVIRONMENT STEPS PER RUN

SAC 1 0.83–0.92 M

PPO 1 1.71–1.88 M

TRAINING HOURS PER RUN

SAC 1 8.1–9.8

PPO 1 0.8–1.0

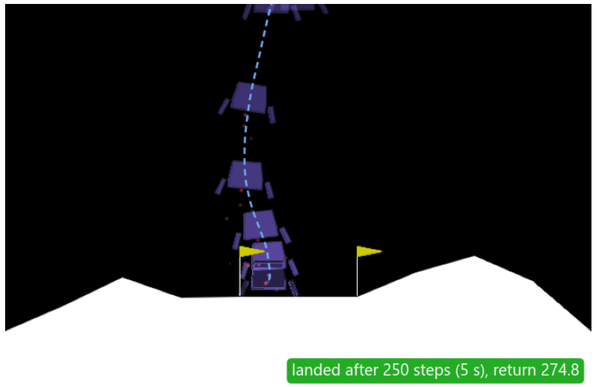
Per episode: the best PPO setting (220, 85% landed) only matches the average SAC setting (218, 82%).

Per sample: SAC 1 needs about half of PPO 1's environment steps.

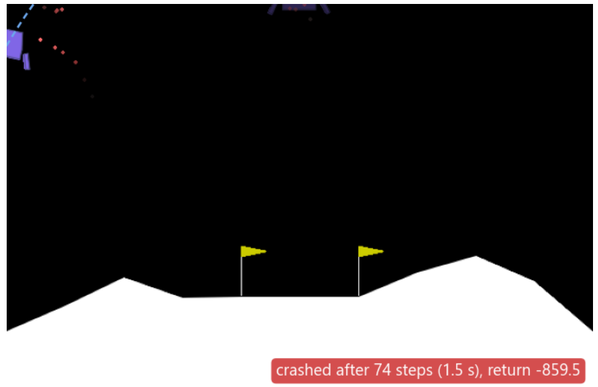
Per hour: a SAC run takes 7–13× as long, since it updates after every step. Early on PPO is the more efficient: 179 (73% landed) after 1000 episodes in ≈17 min, against ≈24 min for SAC 64x64 to reach 160. Later the two are about even (34 vs 43 min).

So: SAC wins per episode and per sample, never per hour. The budget was equal episodes, not equal compute.

F2 · PPO has a learning-rate cut-off and can collapse late



After 2000 episodes: lands in 5 s, return 275



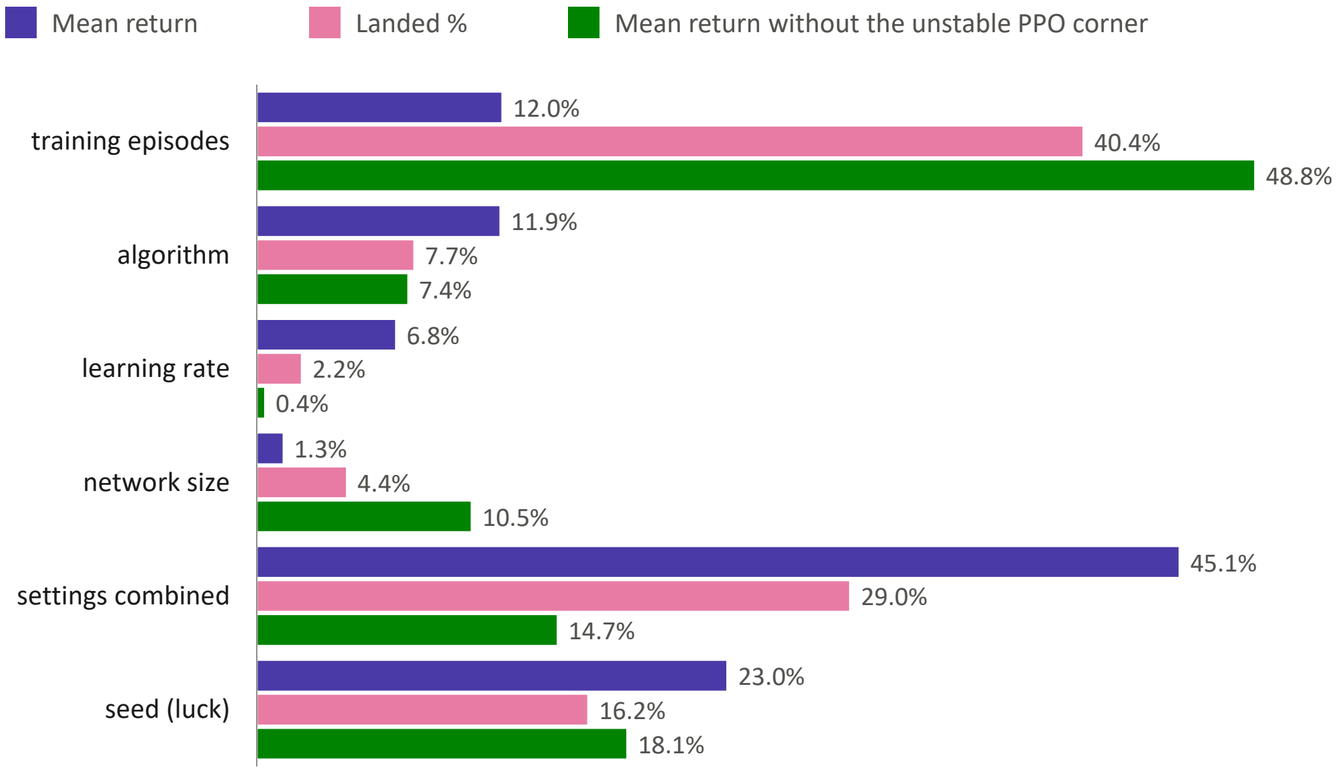
Same run after 4000: off-screen crash in 1.5 s, return −860

169 → −355

mean test return of PPO 256x256 at lr 1e-3, after 2000 → 4000 episodes (72% → 4% landed)

- **A cut-off, not a trend:** lr ≥ 1e-3 breaks PPO on every network above 16x16 (−968 to 83); 1e-4 and 3e-4 reach 175–220.
- **16x16 tolerates every lr** (113–161) and learns fastest with a high one, but never reaches 200.
- **Evidence:** the lr 3e-3 256x256 policies output mean actions up to 58 (valid: ±1), so 99–100% of actions are clipped; their action std grew from 1.0 to 1.6–3.2.
- **Likely cause:** 320 Adam steps per batch at a high constant lr; the clip is no trust region, and an on-policy agent only learns from its own broken behaviour.
- **SAC stays stable** (likely): replay buffer, slow target critics, twin critics, entropy bonus and tanh-squashed actions damp a bad update.

F3 · What matters most depends on the measure



η^2: each setting's share of the spread between all 432 agents (%)

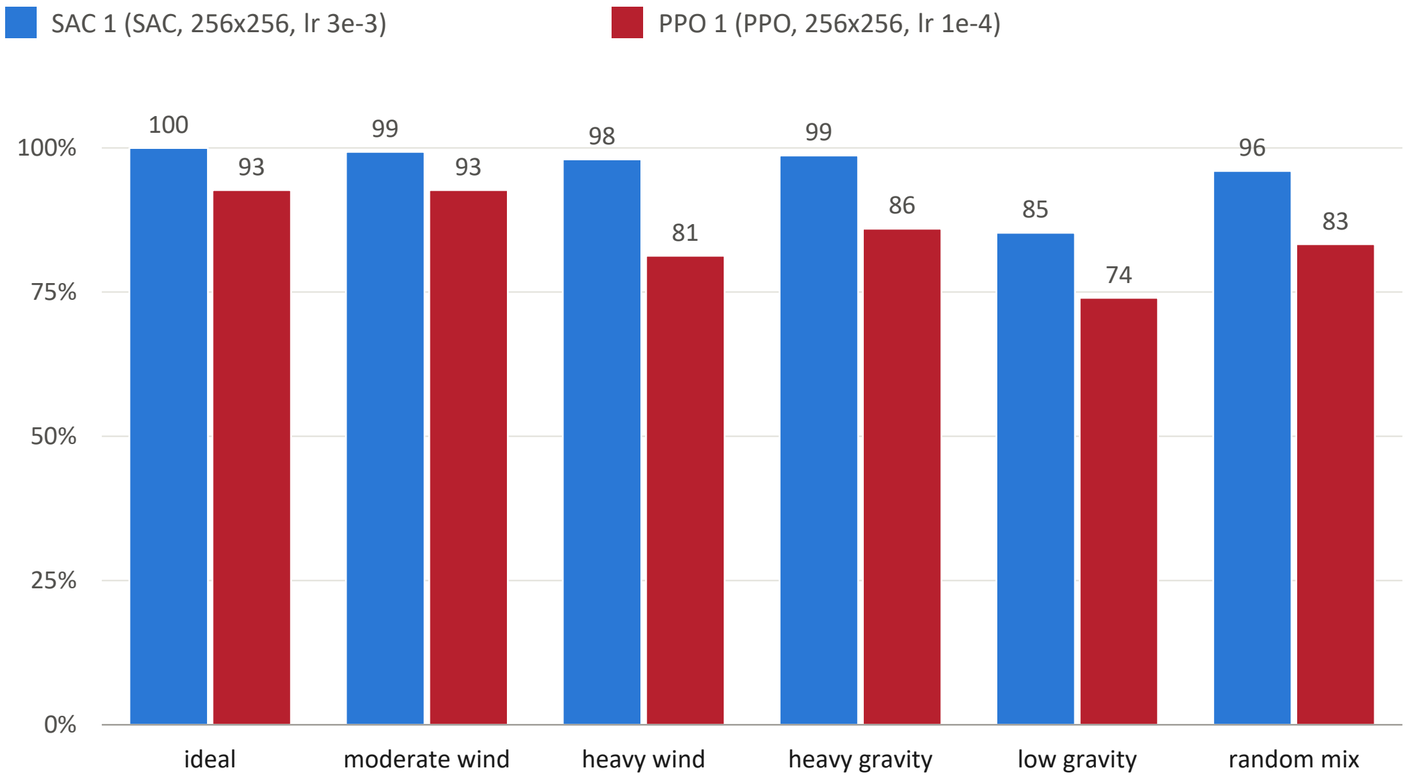
By landed %, which is bounded, **training length matters most: 40%.**

By mean return, PPO collapses near −1000 dominate: combinations explain 45% and seed luck 23%. The unstable corner (PPO, lr ≥ 1e-3, 64x64 or 256x256) is 17% of the agents.

Without that corner, training length explains 49%, and network size rises from 1% to 10.5%: two opposite effects had cancelled out.

Among the 72 final agents, the algorithm matters most (21% of landed %).

F4 · Generalization: low gravity is the hardest



Landed % per test condition after 4000 episodes (150 landings each: 50 planets × 3 seeds)

Low gravity, never seen in training, is hardest: across all settings SAC lands 61% there vs 80–90% elsewhere, PPO 42% vs 46–68%.

PPO crashes, SAC hovers: SAC 1 times out on 10% of low-gravity landings. Likely, without gravity in its state, Earth-like thrust lifts it too much.

Heavy wind widens SAC's lead: PPO 1 drops from 93% to 81% landed, SAC 1 from 100% to 98%.

More training can over-specialize: SAC 64x64, lr 3e-3 loses 20 points from 2000 to 4000 episodes, almost all in low gravity (233 → 164).

Conclusion

1. **SAC beats PPO per episode and per sample:** 275 and 96% landed vs 220 and 85%, with non-overlapping seed ranges.
2. **A cheaper setting, SAC 64x64 at lr 3e-3,** matched SAC 3 after 2000 episodes in about a quarter of the training time.
3. **PPO needs lr ≤ 3e-4 above 16x16** and can collapse late; SAC is insensitive to the learning rate at 64x64 and 256x256.
4. **What matters most depends on the measure:** without the unstable PPO corner, training length explains about half the spread.
5. **Low gravity, never seen in training,** is the hardest condition.

Limitations

- Only 3 seeds per setting; only lr and network size varied
- Equal episodes, not equal compute; most settings not converged
- A 2-D simulator; F2 and F4 explanations are likely, not proven

Future scope

- Train on to 8000 episodes with more seeds; log PPO's KL and clip fraction
- PPO with a learning-rate schedule or a KL target
- Low gravity in training, the planet in the state, or a recurrent network

Thank You

Questions?